[PROGRESS REPORT] π-FORMER : EFFICIENT ZK-SNARKS FOR TRANSFORMERS VIA LEARNABLE LOOKUP ATTENTION

Hideaki Takahashi
Department of Computer Science
Columbia University
ht2673@columbia.edu

ABSTRACT

Deploying large language models (LLMs) in high-stakes settings, legal contracts, medical diagnosis, and financial auditing, demands that *both* the model owner and the user can cryptographically verify that a claimed output was indeed produced by a specific model on a specific input. Zero-Knowledge Succinct Non-interactive Arguments of Knowledge (ZK-SNARKs) offers the right primitive, yet the fundamental complexity of the attention mechanism bottlenecks their application to transformers.

We present π-Former , a transformer architecture and a matching SNARK prover that are *co-designed* from scratch for verifiable inference. Our main contributions are two-fold: (i) *Ternary Linformer Projection*, which restricts the key/value compression matrices to $\{-1, 0, 1\}$, converting complex finite-field multiplicative gates to pure additions; and (ii) *Learnable Lookup Attention* (LLA), a learnable linear-attention variant whose feature map ϕ decomposes into additive sub-tables that plug directly into Lasso lookup arguments.

1 Preliminary

1.1 Transformer Architecture

The Transformer architecture transforms an input sequence of tokens into a sequence of continuous embeddings $\mathbf{X}^{(0)} \in \mathbb{R}^{n \times d}$ [**?**]. The model consists of L identical layers, where each layer l maps the hidden state $\mathbf{H}^{(l-1)}$ to $\mathbf{H}^{(l)}$ through two primary sub-layers: Multi-Head Attention (MHA) and a Position-wise Feed-Forward Network (FFN).

Attention In each layer, the input $\mathbf{X}$ is projected into query, key, and value matrices: $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \mathbf{K} = \mathbf{X}\mathbf{W}_K, \mathbf{V} = \mathbf{X}\mathbf{W}_V$, with weight matrices $\mathbf{W} \in \mathbb{R}^{d \times d}$. The self-attention mechanism computes the compatibility between tokens to produce a weighted context:

$$\mathrm{Att}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\right)\mathbf{V}, \tag{1}$$

where the $\mathrm{softmax}$ operation is applied row-wise. Multi-head attention extends this by performing the operation across h independent heads and concatenating the results.

Activation and Feed-Forward Network The output of the attention mechanism is passed to a Feed-Forward Network (FFN), which typically consists of two linear transformations and a non-linear activation. While earlier models used ReLU, modern LLMs favor Gaussian Error Linear Units (GELU) or SwiGLU variants. GELU is defined as:

$$\mathrm{GELU}(x) = x \cdot \Phi(x) \approx 0.5x\left(1 + \tanh\left(\sqrt{2/\pi}(x + 0.044715x^3)\right)\right), \tag{2}$$

where $\Phi(x)$ is the CDF of the standard normal distribution.

Full Pipeline and Normalization To ensure numerical stability, Layer Normalization (LN) and residual connections are applied around each sub-layer. The complete transformation for a single layer is defined as:

$$\mathbf{X}' = \mathrm{LN}(\mathbf{X} + \mathrm{Att}(\mathbf{X}\mathbf{W}_Q, \mathbf{X}\mathbf{W}_K, \mathbf{X}\mathbf{W}_V)) \tag{3}$$

$$\mathbf{H}^{(l)} = \mathrm{LN}(\mathbf{X}' + \mathrm{GELU}(\mathbf{X}'\mathbf{W}_1)\mathbf{W}_2) \tag{4}$$

where $\mathrm{LN}(\mathbf{x}) = \frac{\mathbf{x}-\mu}{\sqrt{\sigma^2+\epsilon}} \odot \gamma + \beta$. This residual-norm structure allows for the training of very deep networks by preventing signal degradation.

1.2 Optimization for Transformer

Efficient scaling of LLMs requires structural and numerical optimizations to mitigate the quadratic bottlenecks of self-attention and the high memory requirements of model parameters. Current research focuses on approximating the attention matrix and reducing the precision of arithmetic operations to enable deployment on resource-constrained hardware.

Linear Attention To overcome the $O(n^2)$ complexity of standard attention, Linear Attention approximates the attention matrix $P = \mathrm{softmax}(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}})$ via low-rank decomposition. Specifically, let $E, F \in \mathbb{R}^{n\times k}$ be projection matrices where $k \ll n$. The Linformer mechanism computes the context mapping as:

$$\overline{\mathrm{Att}}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{softmax}\left(\frac{\mathbf{Q}(E\mathbf{K})^\top}{\sqrt{d}}\right)(F\mathbf{V}) \tag{5}$$

By computing $E\mathbf{K} \in \mathbb{R}^{k\times d}$ and $F\mathbf{V} \in \mathbb{R}^{k\times d}$ first, the attention operation is reduced to an $O(nk)$ operation. This transformation is theoretically supported by the Johnson-Lindenstrauss lemma, which guarantees that the self-attention matrix can be approximated by a lower-rank matrix with minimal error.

Quantization Quantization strategies reduce the bit-width of weights and activations. Native 1-bit architectures, such as BitNet b1.58 [**?**], utilize ternary weights $w \in \{-1, 0, 1\}$. This removes the need for floating-point multiplications, replacing them with simple additions and subtractions, which drastically reduces energy consumption and latency.

1.3 Verifiable Machine Learning

Verifiable Machine Learning leverages cryptographic protocols to ensure that a given model inference was executed correctly according to a public or committed model specification [**?**]. This is particularly critical in outsourced computation scenarios where a service provider might otherwise return fraudulent or low-quality results to save costs.

Sumcheck Protocol The sumcheck protocol is a cryptographic fundamental for verifying the sum of a v-variate polynomial $g(x_1, \ldots, x_v)$ over a boolean hypercube $\{0,1\}^v$. Given a claimed value C, the prover convinces the verifier that:

$$\sum_{b_1,\ldots,b_v \in \{0,1\}^v} g(b_1, \ldots, b_v) = C \tag{6}$$

In the context of verifiable LLMs, matrix multiplication $C = A \cdot B$ is reduced to a sumcheck over the multilinear extensions (MLE) $\tilde{A}, \tilde{B}$. The verifier only needs to perform $O(v)$ work and a single evaluation of the MLEs, making it highly efficient for verifying large-scale tensor operations.

Lookup Arguments Non-arithmetic operations such as $\mathrm{softmax}$, GELU, and LN are "ZKP-unfriendly" because they involve division or transcendental functions. Lookup arguments (e.g., Lasso) verify that each entry in a secret witness vector $a \in \mathbb{F}^m$ is contained in a public table $T \in \mathbb{F}^N$. Mathematically, this is expressed as verifying the identity of rational functions over a random challenge β:

$$\sum_{i=1}^{m} \frac{1}{\beta + a_i} = \sum_{j=1}^{N} \frac{count_j}{\beta + T_j} \tag{7}$$

where $count_j$ is the number of times T_j appears in a. By utilizing the logarithmic derivative and the sumcheck protocol, this set inclusion check can be performed without explicitly constructing the arithmetic circuits for the underlying non-linear functions.

2 π-Former : Architecture and SNARK Co-Design

π-Former is built around a single design principle: *every operation performed at inference time must map to an arithmetic relation that the underlying SNARK can prove with sub-linear verifier cost.* Prior systems [**?**, **?**] retrofit ZK proofs onto pre-trained softmax transformers, inheriting all of their ZKP-unfriendly operations (exponentiation, division, square root). Instead, we co-design the model architecture and the proof system simultaneously. Our two principal contributions are *Learnable Lookup Attention* (LLA) and *Ternary Linear Projections*: together they reduce the SNARK complexity of a transformer block without sacrificing models' performance.

2.1 Ternary Linear Projections

The Q/K/V/O projections and the FFN layers each require a dense matrix–vector product. In a standard transformer with four $d \times d$ weight matrices per block, this amounts to $4d^2$ multiplications per token per layer, the dominant cost in any finite-field SNARK [**?**]. We eliminate general multiplications from *all* linear layers by restricting weights to a ternary alphabet scaled by a single learnable scalar α, following the BitNet 1.58b design [**?**].

Weight quantization. Given a float weight matrix $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, define a ternary matrix for the sparsity threshold δ

$$\widehat{W}_{ij} = \begin{cases} +1 & W_{ij} > \delta, \\ -1 & W_{ij} < -\delta, \\ 0 & \text{otherwise}, \end{cases} \qquad \widetilde{\mathbf{W}} = \alpha \cdot \widehat{\mathbf{W}}, \tag{8}$$

where $\alpha \in \mathbb{R}_{>0}$ is a per-layer learnable scale. During training we apply the Straight-Through Estimator (STE) [**?**] so gradients flow through the non-differentiable quantization step.

SNARK interpretation. Computing $\mathbf{x}\,\widetilde{\mathbf{W}}^\top = \alpha \cdot (\mathbf{x}\,\widehat{\mathbf{W}}^\top)$ decomposes into two steps: (i) forming $\mathbf{z} = \mathbf{x}\,\widehat{\mathbf{W}}^\top$, which requires only additions and subtractions since every entry of $\widehat{\mathbf{W}}$ lies in $\{-1, 0, +1\}$; and (ii) scaling by α, a single field multiplication broadcast across the output vector. The complete projection circuit therefore contains $O(d_{\text{in}} d_{\text{out}})$ field additions and only $O(d_{\text{out}})$ field multiplications, compared to $O(d_{\text{in}} d_{\text{out}})$ multiplications in a standard dense layer.

We prove the ternary product $\mathbf{z} = \mathbf{x}\,\widehat{\mathbf{W}}^\top$ using the GKR/sumcheck protocol for ML-friendly circuits [**?**], with intermediate matrices committed via Hyrax [**?**] over BN254.

2.2 Learnable Lookup Attention (LLA)

The softmax bottleneck. Standard self-attention (Equation (**??**)) requires n^2 exponentials and n row-wise divisions. Proving these in a SNARK either (i) requires polynomial approximations that introduce thousands of multiplication gates per element, or (ii) requires a monolithic lookup table of size 2^b for b-bit fixed-point values, whose commitment cost is $O(2^b)$ per head [**?**].

Kernel attention. We replace the softmax with a positive-definite kernel $\kappa(\mathbf{q}, \mathbf{k}) = \langle \boldsymbol{\phi}(\mathbf{q}), \boldsymbol{\phi}(\mathbf{k}) \rangle$, where $\boldsymbol{\phi} : \mathbb{R}^{d_h} \to \mathbb{R}^{d_h}$ is a learnable feature map applied element-wise to each scalar entry. This yields

$$\mathrm{LLA}(\mathbf{Q}, \mathbf{K}, \mathbf{V})_t \;=\; \frac{\boldsymbol{\phi}(\mathbf{q}_t)^\top \underbrace{\left(\sum_{s=1}^{n} \boldsymbol{\phi}(\mathbf{k}_s) \mathbf{v}_s^\top \right)}_{\mathbf{M} \;\in\; \mathbb{R}^{d_h \times d_h}}}{\boldsymbol{\phi}(\mathbf{q}_t)^\top \left(\sum_{s=1}^{n} \boldsymbol{\phi}(\mathbf{k}_s) \right)}, \tag{9}$$

where $\mathbf{q}_t, \mathbf{k}_s \in \mathbb{R}^{d_h}$ are per-head query and key vectors. By computing the *context matrix* $\mathbf{M}$ first, the per-token output reduces to the matrix–vector product $\boldsymbol{\phi}(\mathbf{q}_t)^\top \mathbf{M}$, giving $O(n d_h^2)$ total cost per layer rather than $O(n^2 d_h)$. The normalizer $Z_t = \boldsymbol{\phi}(\mathbf{q}_t)^\top \sum_s \boldsymbol{\phi}(\mathbf{k}_s)$ prevents unbounded growth and is computed with no additional table queries.

Additive sub-table decomposition. The crucial ZK-friendly choice is the structure of $\boldsymbol{\phi}$. Rather than adopting a fixed kernel (e.g., $\mathrm{elu}(x)+1$), we parametrize $\boldsymbol{\phi}$ as a *sum of learnable sub-table lookups*. Let b be the quantization bit-width and $c \mid b$ be the decomposition depth, with $\beta = b/c$ bits per chunk. For scalar input x, define the integer index

$$x_{\text{int}} = \mathrm{clamp}\big(\lfloor x/s \rfloor \,,\; 0,\; 2^b - 1 \big)\,, \tag{10}$$

where $s > 0$ is a learned quantization scale. Let the i-th chunk extractor be $\chi_i(x_{\text{int}}) = (x_{\text{int}} \gg i\beta) \,\&\, (2^\beta - 1)$. The feature map is then

$$\boldsymbol{\phi}(x) = \sum_{i=0}^{c-1} T_i[\chi_i(x_{\text{int}})]\,, \tag{11}$$

where $T_0, \ldots, T_{c-1} \in \mathbb{R}^{2^\beta}$ are c learnable sub-tables, each of size 2^β.

Those tables are learnable by treating them as standard model parameters. During the forward pass, the input x is scaled and quantized into a B-bit integer index, which is then decomposed into c chunks that serve as indices for the corresponding sub-tables. To ensure stable convergence, these tables are initialized to approximate a target function like GELU; for instance, the most significant sub-table may be initialized to the GELU shape while the lower-order tables start at zero to prevent initial instability. The quantization and rounding steps are non-differentiable; the architecture employs a STE during backpropagation, which allows gradients to bypass the non-differentiable quantization and flow directly into the sub-table entries, enabling the model to learn an optimal activation function.

Connection to Lasso. Equation (**??**) precisely matches the *structured-table* condition exploited by the Lasso lookup argument [**?**, **?**]: a single query x_{int} to the full 2^b-entry table decomposes into c independent sub-table queries, with the output obtained by summing the c sub-table results.

2.3 End-to-End SNARK Pipeline

The following algorithm outlines the proof generation process for a single π-Former block. It leverages **Lasso** for non-linear lookups, **GKR/Sumcheck** for ternary matrix multiplications, and **Hyrax** for polynomial commitments.

π-Former Transformer Block Prover [1] **Input:** Input commitment $com(X_{in})$; Ternary weight matrices $\hat{W}_Q, \hat{W}_K, \hat{W}_V, \hat{W}_O$; LLA sub-tables $T_{0\ldots c-1}$; Residual scalars α_1, α_2. **Output:** Block proof π_{block}; Output commitment $com(X_{out})$. **Phase 1: Pre-Normalization** (LN_1) Compute integer mean μ and variance σ^2 for X_{in}. Generate **Lasso** range proofs π_{rng} for σ and normalized output Y_1. $com(Y_1) \leftarrow$ Commit to normalized activations. **Phase 2: Batched Ternary Projections** Compute $Q = Y_1\hat{W}_Q^\top, K = Y_1\hat{W}_K^\top, V = Y_1\hat{W}_V^\top$ (Additions only). $com(Q), com(K), com(V) \leftarrow$ Hyrax commitments to projections. Sample challenge $r \in \mathbb{F}$ and run **Batched Sumcheck** for the identity: $\sum_k Y_1(r,k) \cdot (\lambda_Q\hat{W}_Q + \lambda_K\hat{W}_K + \lambda_V\hat{W}_V) = \lambda_Q Q(r) + \lambda_K K(r) + \lambda_V V(r)$. **Phase 3: Learnable Lookup Attention (LLA)** Decompose Q and K into c chunks: $x_{int} \to \{\chi_0, \ldots, \chi_{c-1}\}$. Compute feature maps $\Phi(Q) = \sum T_i[\chi_i(Q)]$ and $\Phi(K) = \sum T_i[\chi_i(K)]$. Prove additive lookup correctness via **Lasso Structured Table Argument** π_{lasso}. $com(\Phi(Q)), com(\Phi(K)) \leftarrow$ Commit to feature maps. **Phase 4: Context and Output Accumulation (GKR Fusion)** Run **Sumcheck** for $M = \Phi(K)^\top V \in \mathbb{F}^{d_h \times d_h}$ over sequence length n. Run **Sumcheck** for $O = \Phi(Q)M \in \mathbb{F}^{n \times d_h}$ over head dimension d_h. *// Optimization: Defer evaluation of* $\Phi(Q), \Phi(K)$ *and batch into one Hyrax opening.* **Phase 5: Output Projection and Residual 1** Compute $O_{proj} = O\hat{W}_O^\top$ and residual $X_{mid} = X_{in} + \alpha_1 O_{proj}$. $com(X_{mid}) \leftarrow com(X_{in}) + \alpha_1 com(O_{proj})$ (Homomorphic addition). **Phase 6: FFN Sub-block** Repeat LN (Step 1-6) and FFN projections using Ternary GKR logic. Compute $X_{out} = X_{mid} + \alpha_2 FFN(X_{mid})$. $\pi_{block} = \{\pi_{rng}, \pi_{proj}, \pi_{lasso}, \pi_{attn}, \pi_{comb}\}, com(X_{out})$.